

CAHIER DES CHARGES

EventFlow

Plateforme de gestion d'événements locaux
Architecture monorepo — 3 applications Next.js

Version	2.2
Date	Juillet 2026
Statut	Validé pour développement
Type de projet	Projet de pratique — architecture monorepo
Applications	Web public · Console Partenaire · Scanner · Console Maître
Backend	API NestJS dédiée — eventflow-api (voir §9.4)

Sommaire

1. Présentation du projet
2. Contexte et étude de l'existant
3. Objectifs et indicateurs de succès
4. Périmètre du projet
5. Acteurs et personas
6. Description des quatre applications
7. Spécifications fonctionnelles détaillées
8. Modèle de données
9. Architecture technique et monorepo
10. Exigences non fonctionnelles
11. Livrables
12. Planning et phases de réalisation
13. Critères d'acceptation et recette
14. Risques et contraintes
15. Glossaire

1. Présentation du projet

EventFlow est une plateforme web de découverte, de gestion et de contrôle d'accès d'événements locaux : concerts, meetups, ateliers, conférences et événements associatifs. Elle s'adresse à trois publics distincts — les visiteurs qui cherchent et réservent des événements, les partenaires (le nom EventFlow des organisateurs) qui les créent et les administrent, et les agents de contrôle qui valident les entrées le jour J.

Le projet poursuit un double objectif : livrer un produit fonctionnel de bout en bout, et servir de terrain d'application rigoureux d'une **architecture monorepo** (3 applications Next.js + packages partagés), conformément aux principes documentés dans le référentiel interne *MONOREPO-GUIDE.md*. Chaque décision de structure devra pouvoir être justifiée par ce guide.

1.1 Vision produit

« Permettre à un partenaire de publier un événement en moins de cinq minutes, à un visiteur de réserver en moins de trois clics, et à un agent de valider une entrée en moins de trois secondes. »

Modèle d'accès : tout visiteur peut devenir partenaire depuis le site public (parcours « Devenir partenaire ») — un seul compte, le rôle s'ajoute et une Organisation est créée. L'accès à la Console Partenaire est régi par un plan : Free, Pro ou Business (§4.4). En v1, seul le plan Free est souscriptible ; la souscription payante arrive en v1.1.

1.2 Périmètre géographique et linguistique

- Interface en **français** (v1) ; structure i18n prête pour l'anglais (v2).
- Événements géolocalisés par ville ; fuseaux horaires gérés côté serveur.
- Monnaie d'affichage : les événements v1 sont gratuits ou à « prix libre » indicatif — pas de paiement en ligne en v1 (voir §4.3).

2. Contexte et étude de l'existant

L'étude des plateformes de référence du marché permet de calibrer les fonctionnalités attendues et d'identifier les standards que les utilisateurs considèrent comme acquis.

Plateforme	Points forts observés	Enseignements pour EventFlow
Eventbrite	Création d'événement guidée, billetterie, page événement riche (SEO), app organisateur avec scan de billets, statistiques de ventes.	Séparer clairement l'app publique de l'app organisateur ; le scan d'entrée est un produit à part entière.
Meetup	Communautés récurrentes, découverte par centres d'intérêt et proximité, RSVP simple sans paiement.	La réservation gratuite (RSVP) suffit pour une v1 ; la découverte par ville/catégorie est le cœur de l'app publique.
Luma (lu.ma)	Pages événement minimalistes et élégantes, inscription en un clic, e-mails automatiques, check-in par QR code.	Un design sobre et rapide crée la confiance ; le QR code de réservation est le pivot entre réservation et check-in.
Billetweb	Outil français, tableau de bord organisateur dense, export des participants, contrôle d'accès multi-agents.	Prévoir l'export CSV des inscrits et la possibilité de plusieurs agents de contrôle par événement.

Positionnement d'EventFlow : une alternative simple et locale, centrée sur les événements gratuits ou communautaires, sans la complexité de la billetterie payante, mais avec la rigueur d'un vrai contrôle d'accès. Le différenciateur du projet est avant tout technique : une architecture monorepo exemplaire et répliquable.

3. Objectifs et indicateurs de succès

3.1 Objectifs produit

- O1 — Publier et découvrir des événements locaux avec une expérience fluide et rapide.
- O2 — Offrir aux partenaires une console complète : création, suivi des réservations, statistiques — dans les limites de leur plan.
- O3 — Fiabiliser l'accueil le jour J grâce à un outil de check-in mobile-first.

3.2 Objectifs techniques

- T1 — Mettre en œuvre un monorepo à 3 apps + packages partagés, sans duplication de code métier.
- T2 — Garantir qu'une nouvelle application s'ajoute sans refactoring structurel — prouvé deux fois : le scanner (P4) puis la console maître (P4b).
- T3 — Centraliser types, schéma de données, authentification et design system dans des packages.

3.3 Indicateurs de succès (KPI)

Indicateur	Cible v1	Mesure
Temps de création d'un événement	< 5 minutes	Test utilisateur chronométré
Parcours de réservation	≤ 3 clics après la page événement	Revue UX
Temps de validation d'une entrée (scan)	< 3 secondes	Test terrain
Parcours « Devenir partenaire »	< 3 minutes	Test utilisateur chronométré
Score Lighthouse (app web, mobile)	≥ 90 en Performance et SEO	CI Lighthouse
Code métier dupliqué entre apps	0 (tout dans packages/)	Revue de code + lint
Couverture de tests des packages partagés	≥ 80 %	Rapport de couverture CI

4. Périmètre du projet

4.1 Inclus dans la version 1

- Quatre applications : **web** (site public), **partner** (Console Partenaire), **scanner** (check-in), **console** (Console Maître super-admin).
- Comptes utilisateurs : visiteur, partenaire, agent de contrôle, super-admin — un seul compte, les rôles s'ajoutent.
- Parcours de conversion « Devenir partenaire » : onboarding court (nom d'organisation, ville) créant une Organisation.

- Système de plans Free / Pro / Business : plans définis, limites du plan Free appliquées (gating serveur), page Tarifs publique. La souscription payante est différée en v1.1.
- Cycle de vie complet d'un événement : brouillon → publié → complet → terminé → archivé.
- Réservation gratuite (RSVP) avec QR code unique par réservation, e-mail de confirmation.
- Check-in par scan de QR code ou saisie manuelle du code court.
- Statistiques partenaire : inscrits, taux de remplissage, taux de présence.
- Export CSV des participants.

4.2 Hors périmètre v1 (envisagé v2+)

- Souscription payante aux plans Pro/Business (Stripe Billing) — planifiée v1.1.
- Billetterie payante des événements (billets vendus aux visiteurs) — v2.
- Applications mobiles natives (iOS/Android).
- Événements récurrents et gestion de communautés type Meetup.
- Notifications push et messagerie interne.
- Multi-langue effectif (la structure i18n est prête, seule la locale fr est livrée).

4.3 Justification du séquençement des plans

Différer l'encaissement (v1.1) tout en livrant le système de plans en v1 donne le meilleur des deux mondes : la charge Stripe (webhooks, factures, downgrades) sort du chemin critique, mais le feature-gating, la page Tarifs et le modèle de données (Organization.plan, champs réservés stripeCustomerId/subscriptionId ; Reservation : prix, devise) sont en place dès le départ — activer le paiement en v1.1 ne demandera aucune migration structurelle. La billetterie payante des événements reste en v2.

4.4 Plans et monétisation

Principe de gating : on ne bride jamais la qualité d'un événement publié ; on bride le volume et les outils de productivité. Toutes les limites sont définies en un point unique et vérifiées côté serveur (TRV-6).

Capacité	Free	Pro	Business
Événements actifs simultanés	2	Illimités	Illimités
Réservations par événement	50	Illimitées	Illimitées
Agents de contrôle par événement	1	5	Illimités
Statistiques	Essentielles	Complètes	Complètes
Export CSV des participants	—	Oui	Oui
E-mail de rappel J-1	—	Oui	Oui
Branding personnalisé (page événement)	—	—	Oui
Membres d'équipe multiples	—	—	Oui (v2)
Prix	0 €	v1 : « bientôt disponible »	v1 : « bientôt disponible »

En v1, tout nouveau partenaire est en Free ; les colonnes Pro/Business sont affichées sur la page Tarifs avec mention « bientôt disponible » et les points d'upgrade sont présents dans la Console (PTN-12). La tarification chiffrée sera fixée en v1.1 avec Stripe Billing.

5. Acteurs et personas

Acteur	Persona type	Besoins principaux	App(s) utilisée(s)
Visiteur	Sarah, 27 ans, cherche des ateliers créatifs dans sa ville le week-end.	Trouver vite, filtrer par ville/date/catégorie, réserver sans friction, retrouver son QR code.	web
Partenaire	Marc, 34 ans, anime une association tech, organise 2 meetups par mois.	Créer/dupliquer un événement, suivre les inscriptions, exporter la liste, mesurer la présence, passer au plan Pro quand son volume grandit.	partner (Console Partenaire)
Agent de contrôle	Léa, bénévole le soir de l'événement, utilise son propre smartphone.	Scanner vite, voir clairement valide/invalid/déjà utilisé, fonctionner avec un réseau instable.	scanner
Super-admin	Vous — propriétaire de la plateforme.	Piloter la plateforme depuis une console dédiée : modération, comptes partenaires et plans, métriques globales.	console (Console Maître)

6. Description des quatre applications

6.1 apps/web — Site public de découverte et réservation

- **Cible** : visiteurs anonymes et connectés. **Priorité** : SEO, vitesse, mobile-first.
- **Rendu** : pages majoritairement statiques/ISR (accueil, listes, pages événement) ; parties dynamiques pour la réservation.
- **Pages clés** : accueil, recherche/filtres, page événement, tunnel de réservation, espace « Mes réservations », pages légales.

6.2 apps/partner — Console Partenaire

- **Cible** : partenaires et super-admin. **Priorité** : productivité, densité d'information, fiabilité.
- **Accès** : 100 % derrière authentification ; aucune page indexable ; rendu dynamique (App Router, Server Components).
- **Écrans clés** : dashboard, CRUD événement, liste des réservations, statistiques, gestion des agents, page Plan (consommation vs limites), paramètres de l'Organisation.
- **Gating** : chaque fonctionnalité limitée vérifie le plan de l'Organisation côté serveur ; l'interface affiche la limite atteinte et le chemin d'upgrade.

6.3 apps/scanner — Application de check-in terrain

- **Cible** : agents de contrôle sur smartphone. **Priorité** : vitesse, lisibilité en conditions réelles, tolérance réseau.
- **Interface** : une seule tâche — scanner ou saisir un code — avec retour visuel plein écran (vert/rouge/orange) et sonore.
- **Spécificité** : PWA installable, cache local de la liste des réservations de l'événement, resynchronisation automatique.

6.4 apps/console — Console Maître (super-admin)

- **Cible** : le super-admin (propriétaire de la plateforme) uniquement. **Priorité** : contrôle, traçabilité, vision globale.
- **Architecture** : conforme au principe « console maître » du monorepo de référence — l'app compose les modules partenaire PLUS un module **platform-admin** (modération, comptes & plans, métriques) que la Console Partenaire n'embarque pas : le code d'administration n'existe pas dans les autres bundles (frontière IP).
- **Écrans clés** : tableau de bord plateforme, gestion des partenaires (Organisations, plans), modération des événements, journal d'audit global.

Règle d'or (issue du guide monorepo) : ces quatre applications ne contiennent que de l'assemblage (pages, layouts, appels). Toute logique réutilisable — types, validation, accès données, composants UI, authentification — vit dans **packages/**.

7. Spécifications fonctionnelles détaillées

Les exigences sont numérotées (WEB-x, PTN-x, SCA-x, CSM-x, TRV-x) et priorisées selon MoSCoW : **M** (Must have), **S** (Should have), **C** (Could have).

7.1 Application web publique (WEB)

ID	P	Exigence
WEB-1	M	L'accueil présente les événements à venir (mis en avant, par date, par ville détectée ou choisie).
WEB-2	M	Recherche avec filtres combinables : ville, catégorie, plage de dates, gratuit/prix libre, places restantes.
WEB-3	M	Page événement : titre, visuel, description riche, date/heure, lieu avec carte, partenaire organisateur, jauge de places, bouton de réservation.
WEB-4	M	Réservation en ≤ 3 clics pour un utilisateur connecté ; création de compte ou connexion intégrée au tunnel sinon.
WEB-5	M	E-mail de confirmation contenant le récapitulatif et le QR code de la réservation.
WEB-6	M	Espace « Mes réservations » : à venir / passées, affichage du QR code, annulation possible jusqu'à H-2.
WEB-7	M	Liste d'attente automatique lorsque l'événement est complet ; promotion automatique en cas d'annulation.
WEB-8	S	Partage social (Open Graph complet) et ajout au calendrier (fichier .ics).
WEB-9	S	Pages partenaire publiques (vitrine de l'Organisation) listant leurs événements.
WEB-10	C	Suggestions « événements similaires » sur la page événement.
WEB-11	M	Page « Tarifs » publique présentant les plans Free / Pro / Business (v1 : Pro et Business marqués « bientôt disponible »).
WEB-12	M	Parcours « Devenir partenaire » : accessible à tout utilisateur connecté ; onboarding court (nom d'organisation, ville) créant l'Organisation en plan Free, puis bascule vers la Console Partenaire.

7.2 Console Partenaire (PTN)

ID	P	Exigence
PTN-1	M	Dashboard : prochains événements, inscriptions des 7 derniers jours, taux de remplissage moyen, actions rapides.
PTN-2	M	Création d'événement en formulaire multi-étapes : infos générales, lieu, date/capacité, visuel, relecture, publication.
PTN-3	M	Cycle de vie : brouillon, publié, complet (automatique), terminé (automatique), archivé, annulé (avec e-mail aux inscrits).
PTN-4	M	Liste des réservations : recherche, statuts (confirmée, annulée, liste d'attente, présente), actions unitaires et export CSV.
PTN-5	M	Statistiques par événement : inscrits vs capacité, courbe d'inscriptions, taux de présence après l'événement.
PTN-6	M	Gestion des agents de contrôle : invitation par e-mail, périmètre limité à un ou plusieurs événements, révocation.

ID	P	Exigence
PTN-7	M	Duplication d'un événement existant (gain de temps pour les récurrents).
PTN-8	—	<i>Déplacé : les fonctions super-admin sont désormais portées par la Console Maître (CSM-1 à CSM-5, §7.4).</i>
PTN-9	S	Journal d'activité par événement (qui a modifié quoi, quand).
PTN-10	C	E-mail de rappel automatique aux inscrits à J-1 (activable par événement — plans Pro et Business).
PTN-11	M	Gating par plan : les limites (événements actifs, réservations par événement, agents) sont appliquées côté serveur, avec message clair et chemin d'upgrade lorsque la limite est atteinte.
PTN-12	S	Page Plan : plan courant de l'Organisation, consommation vs limites, bouton d'upgrade (v1 : « bientôt disponible »).

7.3 Application scanner (SCA)

ID	P	Exigence
SCA-1	M	Connexion agent puis sélection de l'événement du jour parmi ceux autorisés.
SCA-2	M	Scan du QR code via la caméra du téléphone (getUserMedia) avec détection continue.
SCA-3	M	Saisie manuelle d'un code court (8 caractères) en secours du scan.
SCA-4	M	Retour plein écran non ambigu : VERT « Entrée validée + nom », ROUGE « Invalide/annulée », ORANGE « Déjà scannée à HH:MM ».
SCA-5	M	Compteur temps réel : présents / attendus, visible en permanence.
SCA-6	M	Mode hors-ligne : liste des réservations mise en cache au chargement, check-ins stockés localement puis synchronisés.
SCA-7	M	Anti-double-entrée : une réservation scannée sur un appareil est marquée utilisée pour tous les agents (à la resynchronisation en mode dégradé).
SCA-8	S	Recherche d'un participant par nom (arrivé sans QR code).
SCA-9	C	Vibration/bip différenciés selon le résultat du scan.

7.4 Console Maître (CSM)

ID	P	Exigence
CSM-1	M	Tableau de bord plateforme : partenaires actifs, événements publiés, réservations, taux de présence global, tendances sur 30 jours.
CSM-2	M	Gestion des partenaires : liste des Organisations avec plan et consommation, changement de plan manuel (avec recalcul immédiat des limites), suspension/réactivation d'un compte.
CSM-3	M	Modération : file des événements signalés, dépublication avec motif notifié au partenaire par e-mail.
CSM-4	S	Journal d'audit plateforme : consultation et filtrage de l'AuditLog global (acteur, action, entité, période).
CSM-5	C	Vue « en tant que » (lecture seule) d'un compte partenaire, pour le support.

7.5 Exigences transverses (TRV)

ID	P	Exigence
TRV-1	M	Authentification unique : JWT émis par l'API EventFlow (login, refresh, vérification d'e-mail, réinitialisation) ; le monorepo la consomme via packages/auth (descripteurs par app).
TRV-2	M	Autorisation par rôles (visiteur, partenaire, agent, super-admin) appliquée côté serveur dans les trois apps.
TRV-3	M	Double validation : schémas Zod côté front (formulaires) et ValidationPipe stricte (class-validator) côté API — tout champ inconnu est rejeté (400).
TRV-4	M	E-mails transactionnels : confirmation, annulation, promotion de liste d'attente, invitation agent.
TRV-5	S	Traçabilité des actions sensibles (connexions, annulations en masse, changements de rôle ou de plan).
TRV-6	M	Les limites de plan sont définies en un point unique (configuration des plans dans un package partagé) et vérifiées côté serveur — jamais uniquement côté client.

8. Modèle de données

Le schéma est défini une seule fois côté **API NestJS** (MikroORM / PostgreSQL, un domaine hexagonal par entité — cf. §9.4 et BACKEND-GUIDE). Le frontend n'accède jamais à la base : il consomme l'API REST /api/v1, avec types et DTO partagés via **packages/api-contracts**. Entités principales :

Entité	Champs essentiels	Relations et règles
User	id, email (unique), nom, hash mot de passe, rôle global, emailVerifiedAt, createdAt.	1-n Reservations ; 1-n Organizations (propriétaire) ; n-n Events via EventAgent (en tant qu'agent).
Organization	id, nom, slug (unique), ville, plan (FREE/PRO/BUSINESS), planStatus, ownerId, createdAt ; champs réservés v1.1 : stripeCustomerId, subscriptionId.	1-n Events ; appartient à un User (owner) ; membres multiples en v2. Créée par le parcours « Devenir partenaire » (WEB-12). Le plan porte les limites de gating (§4.4).
Event	id, slug (unique), titre, description, catégorie, ville, adresse, lat/lng, startsAt, endsAt, capacité, statut, visuelUrl, prixLibre, organizationId.	Statuts : DRAFT, PUBLISHED, FULL, FINISHED, ARCHIVED, CANCELLED. FULL et FINISHED sont calculés/automatiques.
Reservation	id, code court (8 car., unique), qrToken (signé), statut, userId, eventId, createdAt, cancelledAt, champs réservés v2 : prix, devise, paymentStatus.	Statuts : CONFIRMED, WAITLISTED, CANCELLED, CHECKED_IN. Unicité (userId, eventId) : une réservation active par personne.
CheckIn	id, reservationId (unique), agentId, scannedAt, device, syncedAt (null si créé hors-ligne non synchronisé).	L'unicité sur reservationId garantit l'anti-double-entrée au niveau base de données.
EventAgent	eventId, userId, invitedAt, revokedAt.	Table de liaison définissant le périmètre d'un agent.
AuditLog	id, actorId, action, entité, entityId, metadata (JSON), createdAt.	Alimente le journal d'activité (ADM-9, TRV-5).

Règles d'intégrité clés : la jauge se calcule en base (transactions) et jamais côté client ; le passage FULL est déclenché atomiquement à la dernière place ; le qrToken est signé (HMAC) pour être vérifiable même hors-ligne par l'app scanner. Les limites de plan (événements actifs, réservations) sont vérifiées dans les mêmes transactions que les créations qu'elles contraignent.

9. Architecture technique et monorepo

9.1 Stack technique

Domaine	Choix	Justification
Frontend	Next.js (App Router) × 4 apps	SSG/ISR pour web ; dynamique pour partner, console et scanner.
Monorepo front	npm workspaces — aucun Turbo/Nx/pnpm	Conformité MONOREPO-GUIDE (P1) : module = package, app = composition.
Backend	API NestJS 11 sur Fastify — dépôt dédié eventflow-api	Architecture hexagonale par domaine, conforme à BACKEND-GUIDE.md (§9.4).

Domaine	Choix	Justification
Langage	TypeScript strict de bout en bout	Contrats front/back partagés via packages/api-contracts.
Base de données	PostgreSQL + MikroORM (migrations versionnées)	Le schéma vit côté API ; le frontend n'accède jamais à la base.
Authentification	JWT Bearer émis et vérifié par l'API EventFlow	Le monorepo la consomme via packages/auth (descripteurs) + rewrites.
Plans & gating	@RequiresPlan + PlanGuard côté API ; limites en configuration unique	Un point unique définit Free/Pro/Business ; vérification serveur (TRV-6).
Jobs & cache	BullMQ + Redis	E-mails asynchrones, promotion de liste d'attente, resynchronisation des check-ins.
UI	Tailwind CSS + design system @eventflow (tokens + composants)	Thème et composants uniques pour les 4 apps.
Validation	class-validator (ValidationPipe globale stricte) côté API ; Zod côté front	Double barrière : formulaire (front) et contrat (API — champ inconnu rejeté en 400).
E-mails	Envoyés par l'API via jobs BullMQ, templates versionnés	Confirmations, annulations, promotions, invitations d'agents (TRV-4).
Docs API	Swagger / OpenAPI	Chaque endpoint documenté et testable (@ApiTags, @ApiBearerAuth).
Qualité	API : ESLint + Prettier + Jest · Front : ESLint + Vitest + Playwright	Unitaires (domaines), intégration (API), E2E (parcours complets).
CI/CD	Build par app front ; API conteneurisée (Docker), migrations en pipeline	Déploiements indépendants ; migrate:up avant démarrage de l'API.

9.2 Structure du monorepo

```

eventflow/
+-- apps/
|   +-- web/           # Site public (SEO, réservation)
|   +-- partner/       # Console Partenaire
|   +-- scanner/       # Check-in PWA mobile-first
|   `-- console/       # Console Maître (compose partner + platform-admin)
+-- packages/
|   +-- ui/            # Design system (boutons, cartes, badges...)
|   +-- api-contracts/ # Types, DTO et client HTTP typé de l'API EventFlow
|   +-- auth/          # Descripteurs d'auth JWT (API), composés par app
|   +-- validators/    # Schémas Zod + types inférés
|   `-- config/        # tsconfig, eslint, tailwind partagés
+-- scripts/scaffold-routes.mjs # Codegen des routes (MONOREPO-GUIDE P10)
+-- plopfile.mjs + plop-templates/
`-- package.json       # Scripts racine uniquement

```

9.3 Règles d'architecture (contractuelles)

- **R1** : une app n'importe jamais de code d'une autre app ; uniquement depuis packages/.
- **R2** : les packages ne dépendent pas des apps ; les dépendances vont dans un seul sens (apps → packages).

- **R3** : le frontend n'accède JAMAIS à la base : toute donnée passe par l'API /api/v1 via packages/api-contracts ; aucun ORM dans le monorepo.
- **R4** : tout composant UI utilisé par ≥ 2 apps est promu dans packages/ui.
- **R5** : les versions de dépendances communes (react, next, zod...) sont épinglées à la racine du monorepo (npm workspaces, versions exactes pour react/next).
- **R6** : chaque package expose une API publique via son index ; pas d'import de chemins internes profonds.
- **R7** : toute exception à ces règles doit être documentée dans MONOREPO-GUIDE.md avant d'être commitée.

9.4 Backend — API NestJS dédiée (eventflow-api)

- **Dépôt séparé** du monorepo frontend, construit selon BACKEND-GUIDE.md : NestJS 11 sur Fastify, architecture hexagonale (Ports & Adapters) organisée par domaine — un domaine par entité métier : partner (Organisations), event, booking (réservations), check-in, plan.
- **Contrat de couture**, consommé par les rewrites du monorepo (pattern next-config.ts + auth-descriptor du MONOREPO-GUIDE) : routes sous /api/v1 ; CORS credentials avec les origines des 4 apps ; auth Bearer JWT ; erreurs en enveloppe { error: { type, code, message, param } } ; succès renvoyés bruts (pas d'enveloppe data).
- **Gating serveur** : décorateur @RequiresPlan + PlanGuard, limites lues depuis la configuration unique des plans (TRV-6, PTN-11) et appliquées dans les transactions (jauge, quotas d'événements actifs).
- **Check-in hors-ligne** : endpoints idempotents (clé d'idempotence par scan, code idempotency_key_in_use) + synchronisation batch rejouée via BullMQ (SCA-6/SCA-7).
- **Compléments vs projet de référence** (recommandations issues du scan) : validation du .env (validationSchema), rate-limiting (@nestjs/throttler), helmet, healthcheck (@nestjs/terminus), conteneurisation Docker.

10. Exigences non fonctionnelles

Catégorie	Exigence	Cible mesurable
Performance	Pages publiques rapides sur mobile 4G.	LCP < 2,5 s ; Lighthouse Perf ≥ 90 ; réponse API scan < 300 ms.
Disponibilité	Le scanner reste utilisable si le réseau tombe pendant l'événement.	Mode hors-ligne fonctionnel ≥ 30 min ; zéro perte de check-in.
Sécurité	Protection des comptes et des données.	Hash bcrypt ; JWT Bearer courte durée + refresh ; rate-limiting (@nestjs/throttler) sur login et réservation ; helmet ; qrToken signé HMAC non rejouable.
RGD	Respect de la réglementation européenne.	Consentement explicite, page de confidentialité, export et suppression du compte sur demande, minimisation des données.
Accessibilité	Utilisable au clavier et lecteur d'écran.	Conformité WCAG 2.1 niveau AA sur les parcours critiques.
SEO	Événements découvrables sur les moteurs.	Rendu serveur, balises Open Graph, données structurées schema.org/Event, sitemap dynamique.
Compatibilité	Navigateurs récents.	Deux dernières versions majeures de Chrome, Firefox, Safari, Edge ; scanner testé iOS Safari + Android Chrome.
Maintenabilité	Un nouveau développeur est productif rapidement.	Installation en ≤ 3 commandes documentées ; README par app et par package.
Scalabilité	Tenir un pic d'inscriptions.	500 réservations simultanées sans erreur de jauge (test de charge).

11. Livrables

- **L1** — Le monorepo Git complet (apps + packages) conforme aux règles du §9.3.
- **L2** — Les trois applications déployées en production (URLs distinctes) + environnement de préproduction.
- **L3** — Le schéma de base de données avec migrations et jeu de données de démonstration (seed).
- **L4** — La suite de tests : unitaires (packages), intégration (API), bout-en-bout (parcours réservation et check-in).
- **L5** — La documentation : README racine, README par app/package, MONOREPO-GUIDE.md mis à jour, guide de déploiement.
- **L6** — Le pipeline CI/CD : lint + typecheck + tests + build par app front ; API conteneurisée avec migrations exécutées en pipeline.
- **L7** — Le dépôt eventflow-api : domaines complets (partner, event, booking, check-in, plan), migrations, Swagger, tests — conforme à BACKEND-GUIDE.md.

12. Planning et phases de réalisation

Phase	Durée	Contenu	Jalon de sortie
P0 — Fondations	1 sem.	Initialisation du monorepo front (packages config/ui/validators/api-contracts) ET du dépôt eventflow-api (checklist BACKEND-GUIDE), CI de base, conventions.	« npm run dev » lance une app vide stylée ; lint/typecheck verts en CI.
P1 — Socle métier	2 sem.	Domaines API (partner, event, booking) : entités MikroORM, migrations, use-cases de base ; auth JWT (login/refresh) ; seed de démo ; packages/auth et api-contracts côté front ; premiers composants UI.	Connexion fonctionnelle ; base seedée ; Storybook ou page vitrine des composants.
P2 — App web	2 sem.	Accueil, recherche/filtres, page événement, tunnel de réservation, e-mails, espace réservations, page Tarifs, parcours « Devenir partenaire ».	Parcours visiteur complet en préproduction (WEB-1 à WEB-7, WEB-11, WEB-12).
P3 — Console Partenaire	2,5 sem.	Dashboard, CRUD événement, réservations, statistiques, gestion des agents, export CSV, gating des plans, page Plan.	Un partenaire gère un événement de bout en bout, dans les limites de son plan (PTN-1 à PTN-7, PTN-11).
P4 — App scanner	1,5 sem.	PWA, scan caméra, saisie manuelle, écrans de résultat, mode hors-ligne, synchronisation.	Check-in testé en conditions réelles sur 2 téléphones (SCA-1 à SCA-7).
P4b — Console Maître	0,5 sem.	Module platform-admin (CSM-1 à CSM-4) + app console composant les modules requis ; second test d'extensibilité de l'architecture.	Le super-admin pilote partenaires, plans et modération depuis apps/console.
P5 — Durcissement	1 sem.	Tests E2E, test de charge jauge, accessibilité, SEO, RGPD, revue des règles monorepo.	Recette complète (§13) validée ; v1.0 taguée et déployée.

Durée totale estimée : 10,5 semaines pour un développeur solo à temps partiel soutenu, phases séquentielles. L'ajout du scanner en P4 (dernière app) sert volontairement de test de l'extensibilité de l'architecture (objectif T2).

13. Critères d'acceptation et recette

La version 1 est réputée conforme lorsque les scénarios suivants passent en préproduction, sans intervention manuelle :

#	Scénario de recette	Résultat attendu
S1	Un partenaire crée, publie puis duplique un événement de 50 places.	Événement visible sur le site public en < 1 min ; duplicata en brouillon.
S2	Une visiteuse réserve, reçoit l'e-mail, annule, re-réserve.	QR code reçu ; place libérée puis reprise ; jauge exacte à chaque étape.
S3	51 personnes tentent de réserver 50 places (test de charge).	50 confirmées, 1 en liste d'attente ; aucune sur-réservation.
S4	Une inscription en liste d'attente est promue après une annulation.	Promotion automatique + e-mail ; statuts cohérents dans l'admin.
S5	Le jour J, deux agents scannent en parallèle, dont un passe hors-ligne 10 min.	Aucune double entrée ; check-ins hors-ligne synchronisés ; compteur final exact.

#	Scénario de recette	Résultat attendu
S6	Un QR code déjà utilisé, puis un QR falsifié, sont présentés au scanner.	Écran orange « déjà scanné » avec l'heure ; écran rouge « invalide ».
S7	Un utilisateur demande l'export puis la suppression de son compte.	Export livré ; données personnelles supprimées/anonymisées, réservations historiques agrégées.
S8	Revue d'architecture : recherche d'imports interdits et de code dupliqué.	Zéro violation des règles R1 à R6 ; rapport de lint architecture vert.
S9	Un visiteur connecté devient partenaire puis publie son premier événement.	Organisation créée en plan Free ; bascule vers la Console ; événement visible sur le site public.
S10	Un partenaire Free tente de dépasser une limite : 3e événement actif, puis 51e réservation.	Blocage propre côté serveur, message avec chemin d'upgrade ; aucune limite contournable en manipulant le client.
S11	Depuis la Console Maître : passage d'une Organisation en Pro, puis dépublication d'un événement signalé.	Limites recalculées immédiatement ; événement retiré du site public ; partenaire notifié ; les deux actions tracées dans l'AuditLog. Le bundle partner ne contient aucun code platform-admin (marqueur de bundle).

14. Risques et contraintes

Risque	Impact	Prob.	Mitigation
Dérive du périmètre (ajout de la billetterie payante en cours de route).	Élevé	Moyenne	Périmètre v1 contractuel (\$4) ; toute demande hors périmètre passe en backlog v2.
Contournement des limites de plan (gating) via le client ou l'API.	Élevé	Moyenne	Limites définies en un point unique et appliquées côté serveur/base (TRV-6) ; scénario S10 obligatoire en recette.
Concurrence d'accès sur la jauge (sur-réservation).	Élevé	Moyenne	Transactions atomiques en base, contrainte d'unicité, scénario de charge S3 obligatoire.
Réseau instable le jour J rendant le scanner inutilisable.	Élevé	Élevée	Mode hors-ligne SCA-6/SCA-7 conçu dès le départ, testé en conditions réelles (S5).
Érosion des règles monorepo au fil du développement.	Moyen	Élevée	Lint d'architecture en CI (imports interdits), revue systématique via MONOREPO-GUIDE.md.
Compatibilité caméra sur iOS Safari (scan QR).	Moyen	Moyenne	Bibliothèque de scan éprouvée, saisie manuelle SCA-3 comme secours permanent.
Développeur solo : perte de rythme.	Moyen	Moyenne	Phases courtes avec jalons démontrables ; chaque phase livre quelque chose d'utilisable.

15. Glossaire

Terme	Définition
Monorepo	Dépôt Git unique contenant plusieurs applications et packages qui partagent outillage, types et code.

Terme	Définition
Workspace	Sous-projet du monorepo (une app ou un package) déclaré dans les workspaces du package.json racine.
Package partagé	Module interne (ui, database, auth...) consommé par les applications, jamais publié sur npm.
Partenaire	Utilisateur ayant activé le rôle d'organisateur via « Devenir partenaire » ; il agit au nom d'une Organisation.
Console Maître	Application réservée au super-admin (apps/console) : elle compose les modules partenaire plus le module platform-admin.
eventflow-api	Backend NestJS dédié (dépôt séparé) : architecture hexagonale par domaine, seule porte d'accès à la base de données.
Enveloppe d'erreur	Format contractuel des erreurs de l'API : { error: { type, code, message, param } } ; les succès sont renvoyés sans enveloppe.
Organisation	Entité propriétaire des événements d'un partenaire ; porte le plan (Free/Pro/Business) et ses limites.
Plan / Gating	Niveau d'abonnement d'une Organisation et mécanisme d'application serveur de ses limites.
Freemium	Modèle où le plan gratuit est pleinement utilisable, les plans payants ajoutant volume et productivité.
RSVP	Réservation gratuite confirmant l'intention de venir, sans paiement.
Check-in	Validation de l'entrée d'un participant le jour de l'événement.
Jauge	Nombre de places disponibles ; pilotée en base de données de façon atomique.
Liste d'attente	File des demandes reçues après épuisement de la jauge, promue automatiquement.
PWA	Progressive Web App : application web installable, capable de fonctionner hors-ligne.
ISR	Incremental Static Regeneration : pages statiques Next.js régénérées à intervalle défini.
MoSCoW	Méthode de priorisation : Must / Should / Could / Won't have.
Recette	Phase de vérification formelle de la conformité du produit au présent cahier des charges.

Fin du document — EventFlow, cahier des charges v2.2 (v2 : Partenaire, Organisation, plans + gating ; v2.1 : Console Maître ; v2.2 : backend API NestJS dédiée conforme à BACKEND-GUIDE.md, stack monorepo alignée sur MONOREPO-GUIDE.md). Toute modification du périmètre ou des règles d'architecture doit faire l'objet d'un avenant versionné.